

Report
Data Mining II

Davide Rizzello N° 689583

September 1, 2025

Contents

1	Data Understanding & Preparation	3
1.1	Tabular Data	3
1.2	Time Series	4
2	Time Series Advanced Analysis	6
2.1	Clustering	6
2.2	Motifs and Discords Discovery	7
2.2.1	Selected Time Series	7
2.2.2	Motifs & Discords	8
2.3	Classification	10
2.3.1	KNeighborsTimeSeriesClassifier	10
2.3.2	1D Convolutional Neural Network	10
2.3.3	(Mini)ROCKET + Ridge	11
2.4	Shapelet Transform + Random Forest	12
3	Advanced Pre-Processing	14
3.1	Outlier Detection	14
3.1.1	Methodology	14
3.1.2	Dimensionality Reduction and Visualization	15
3.1.3	Outliers Management	15
3.2	Imbalanced Learning	16
3.2.1	UnderSampling	17
3.2.2	OverSampling	17
3.2.3	Conclusions	18
4	Advanced Machine Learning	19
4.1	Advanced Classification	19
4.1.1	Baseline Model: Perceptron	19
4.2	Multi-Layer Perceptron (MLP)	20
4.2.1	Model Architecture	20
4.2.2	Hyperparameter Optimization and Final MLP Model	21
4.3	Ensemble methods	23
4.3.1	Random Forest	23
4.3.2	Boosting Methods	24
4.4	Advanced Regression	26
4.4.1	Random Forest Regressor	27
4.4.2	LightGBM Regressor	27
4.4.3	Results and Comparison	27

5	Explainable AI	28
5.0.1	Global Explanations	28
5.0.2	Local Explanations	30
5.0.3	Conclusion	31

1 Data Understanding & Preparation

1.1 Tabular Data

The primary objective of this project was to analyze and preprocess a tabular dataset derived from the Internet Movie Database (IMDB), consisting of 149,531 rows and 32 columns. The dataset includes a diverse range of metadata related to audiovisual content, including title-level attributes, audience engagement metrics, critical rating indicators, and production details.

An initial inspection (E.D.A.) was conducted. Firstly it was assessed the extent of missing values. Variables such as runtimeMinutes, endYear, and regions exhibited substantial missingness. The runtimeMinutes variable was imputed using the median value grouped by titleType, while endYear and regions were excluded due to their high proportion of nulls. For categorical fields like countryOfOrigin and genres, missing values were filled with 'Unknown' to keep the categories consistent and ready for encoding.

A significant number of duplicate entries were identified based on the originalTitle field, totaling 12,126 duplicates. To resolve this, the dataset was sorted by originalTitle, startYear, and numVotes in descending order of popularity. Only the most voted version was retained per duplicated group, reducing the dataset size by 3,080 rows and ensuring a more representative set of unique titles.

Multi-valued categorical attributes were parsed and transformed through custom list-parsing functions. The genres field, often stored as stringified lists or comma-separated strings, was standardized and filtered to retain only the ten most frequent genres. Entries containing only rare or missing genres were mapped to "Unknown". These lists were subsequently encoded using MultiLabelBinarizer, resulting in binary indicator variables with column names in the format genre_X. The same methodology was applied to the soundMixes field.

The countryOfOrigin field was also cleaned and parsed into lists. To enrich the dataset with macro-geographic information, each country was mapped to its respective continent using a manually defined ISO code map. However, not all country codes were covered by the static map, resulting in missing continent labels. To address this, a large language model (gemma-3-27b-it) was employed to infer missing continent information using a prompt that included each title's name, release year, and number of award wins. The model returned one of seven predefined continents in a structured format.

Evaluation of the LLM-based predictions against known continent labels (when available) was conducted using standard classification metrics, yielding satisfactory precision and recall. The predicted continent labels were then injected into the dataset, completing the geographic enrichment process.

Continent	Precision	Recall	F1-Score	Support
Africa	0.11	0.25	0.15	8
Asia	0.64	0.74	0.68	133
Europe	0.60	0.68	0.64	284
North America	0.78	0.73	0.76	489
Oceania	0.44	0.16	0.24	25
Other	0.00	0.00	0.00	48
South America	0.24	0.77	0.36	13
Accuracy			0.66	1000
Macro avg	0.40	0.47	0.40	1000
Weighted avg	0.65	0.66	0.65	1000

Table 1.1: Classification report for LLM-based continent prediction.

Several new features were engineered to enhance the dataset’s information density. These included `title_length` and `word_count` (based on title text), `num_countries` (count of producing countries), and `award_ratio` (wins over nominations). Additionally, `total_multimedia` was created as the sum of `totalImages` and `totalVideos`.

To further enrich the dataset with textual features, two additional attributes were derived from the `originalTitle` field. First, the number of words in each title was computed and stored in a new column `word_count`, serving as a basic proxy for title complexity.

Second, sentiment analysis was conducted on the title strings using the `TextBlob` library. Based on this continuous score, a categorical sentiment label was assigned as follows: entries with polarity greater than 0.05 were labeled as "Positive", those below -0.05 as "Negative", and all others as "Neutral".

Although the original `originalTitle` field was subsequently removed from the dataset due to its high cardinality and limited modeling utility in raw form, the engineered features `word_count`, and sentiment were retained for their potential contribution to predictive performance.

Outliers were visually inspected using boxplots across numerical variables such as `runtimeMinutes`, `awardWins`, and `numVotes`. Although not removed at this stage, their identification was noted for model robustness considerations.

As a final preprocessing step, all remaining categorical variables were one-hot encoded. Numerical variables were scaled using `StandardScaler` to ensure zero mean and unit variance, particularly important for distance-based or gradient-based models.

The resulting dataset was fully cleaned, enriched, encoded, and scaled—comprising 116,539 rows and 63 columns—thus rendering it structurally consistent and semantically ready for downstream classification, regression, and outlier detection tasks.

1.2 Time Series

In this section, it is described the preprocessing and preparation steps carried out on a dataset of time series derived from box office performance data. Each time series represents the daily domestic gross revenue of a movie, starting from its theatrical release date. The dataset comprises approximately 1,000 time series, each aligned by release date and indexed by day number rather than calendar date. Additionally, metadata such as movie genres and average viewer ratings are associated with each series.

Initially, an overview of the raw data was provided by plotting time series across different rating categories and genres (See figure 1.1).

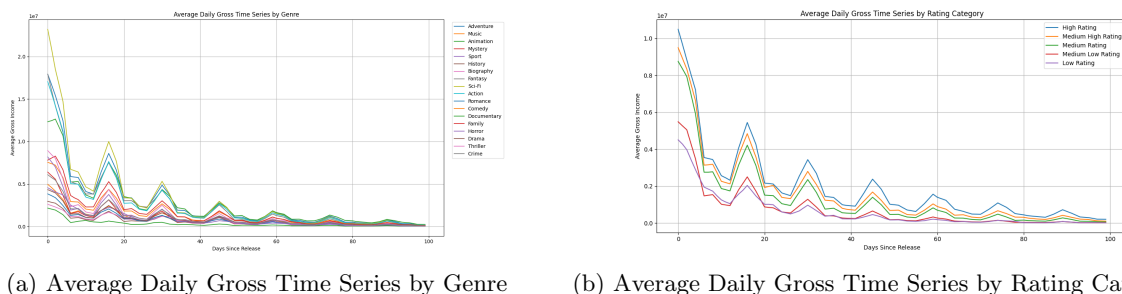


Figure 1.1: Average Daily Gross Time Series

Prior to performing downstream tasks such as clustering or classification, several preprocessing operations were applied to the time series dataset. These transformations were aimed at improving the quality of the signals, enhancing comparability, and enabling more robust learning procedures. The four main preprocessing steps include outlier detection, normalization, detrending, and dimensionality reduction via Piecewise Aggregate Approximation (PAA).

Outliers within each time series were detected using the Interquartile Range (IQR) method in a cumulative fashion. The classical IQR rule defines an observation as an outlier if it lies outside the range defined by:

$$[Q1 - k \cdot IQR, Q3 + k \cdot IQR],$$

where $Q1$ and $Q3$ denote the first and third quartiles, respectively, and $IQR = Q3 - Q1$. A constant $k = 1$ was adopted as a standard threshold.

To capture the temporal emergence of outliers, the method was extended to a recursive setting: starting from an initial window (of size 14), the outlier status of each new point was evaluated by computing the IQR boundaries using all previous observations. This dynamic labeling allows for the detection of anomalous revenue spikes or drops over time, potentially corresponding to external factors such as marketing campaigns or awards.

The variability in revenue magnitudes across movies necessitated a normalization step to make the time series comparable in scale. Each series was individually normalized using z-score standardization:

$$z_t = \frac{x_t - \mu}{\sigma},$$

where x_t is the raw value at time t , and μ , σ are the mean and standard deviation of the series. This transformation ensures that each time series has zero mean and unit variance, thus preventing magnitude differences from dominating similarity calculations or classification criteria.

To isolate fluctuations of interest from underlying trends, a detrending operation was applied using the Detrender module from the sktime library. This transformation removes global linear trends from each time series, thereby stabilizing the mean over time (Figure 1.2b). Detrending is particularly useful in performance series like box office revenues, where long-term drifts (e.g., due to growing popularity) may obscure short-term variations relevant to predictive modeling.

The final preprocessing step involved the application of Piecewise Aggregate Approximation (PAA), a widely used method for reducing the dimensionality of time series while preserving their overall shape. Each time series was divided into equal-length segments, and the mean of each segment was used to replace its values, resulting in a compressed representation (Figure 1.2a).

This approximation facilitates faster computation and reduces noise, while retaining the series' main structural properties. The reduced series can then be used in tasks such as clustering or shape-based classification.

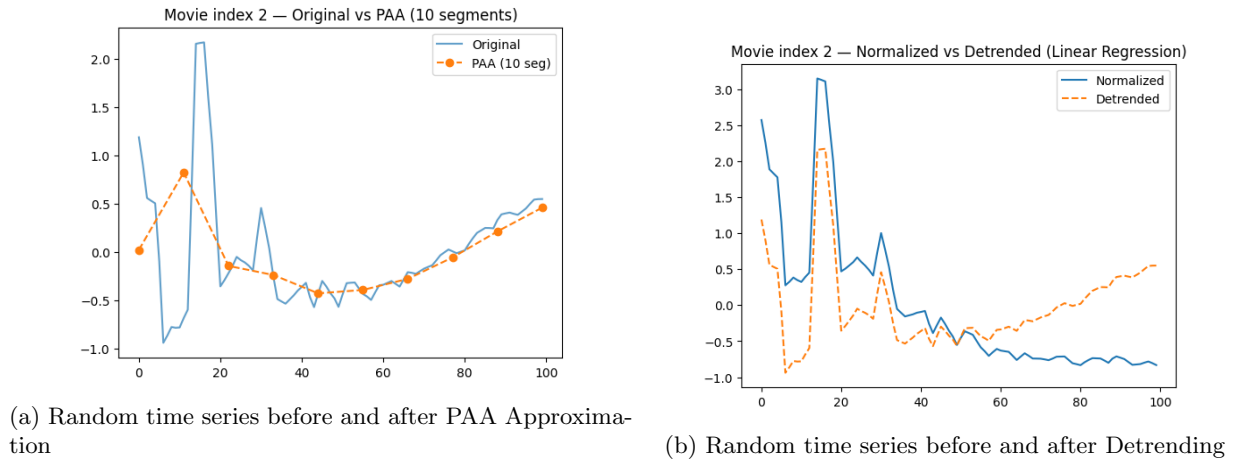


Figure 1.2: Examples of Time Series Pre-processing

2 Time Series Advanced Analysis

2.1 Clustering

This section describes the application of unsupervised clustering techniques to time series data representing box office performance over time. The objective is to group movies based on similarities in their revenue trajectories using two different distance metrics: Euclidean distance and Dynamic Time Warping (DTW). The analysis investigates whether patterns in the revenue evolution—such as early peaks, sustained performance, or delayed growth—can be automatically discovered through clustering.

Clustering was conducted using the TimeSeriesKMeans algorithm from the sktime library. The following settings and strategies were adopted:

- Distance Metrics: Both Euclidean distance and Dynamic Time Warping (DTW) were used as similarity measures to explore their effectiveness in capturing temporal patterns.
- Number of Clusters: Based on preliminary experiments and elbow plots, the number of clusters was fixed at $k = 4$ for comparative purposes.
- Fitting: Two clustering models were trained independently—one with Euclidean distance and one with DTW—on the PAA-transformed data.
- Evaluation: For each distance metric, silhouette and SSE scores were computed.

A 4-cluster solution ($k = 4$) was selected based on silhouette analysis and interpretability of the resulting groups. Each movie was thus assigned to one of four clusters reflecting the shape of its revenue time series. A silhouette score analysis revealed that DTW achieves slightly higher values than Euclidean distance for most k values, indicating that it forms more compact and well-separated clusters. As a result, DTW was adopted for all subsequent analyses.

The time series within each cluster were plotted to visually assess intra-cluster similarity, highlighting how well the grouping algorithm aligned sequences with shared temporal evolution.

The visual inspection suggests that the DTW-based clustering captures distinct behavioral trends in revenue progression, including early peaks, slow rises, and long-tail patterns. The distribution of time series across the four clusters is as follows:

- Cluster 0: 212 time series
- Cluster 1: 58 time series
- Cluster 2: 151 time series
- Cluster 3: 713 time series

These counts indicate a relatively dominant cluster (Cluster 3), suggesting a common revenue trajectory shared by the majority of movies, with other clusters capturing more niche or distinct temporal patterns.

To investigate potential associations between movie genres and revenue patterns bar plots and box plots were generated to visualize the distribution of genres and rating across clusters. The analysis suggests that certain genres (e.g., horror, animation, or action) are disproportionately represented in specific clusters, potentially indicating distinct temporal revenue dynamics associated with genre.

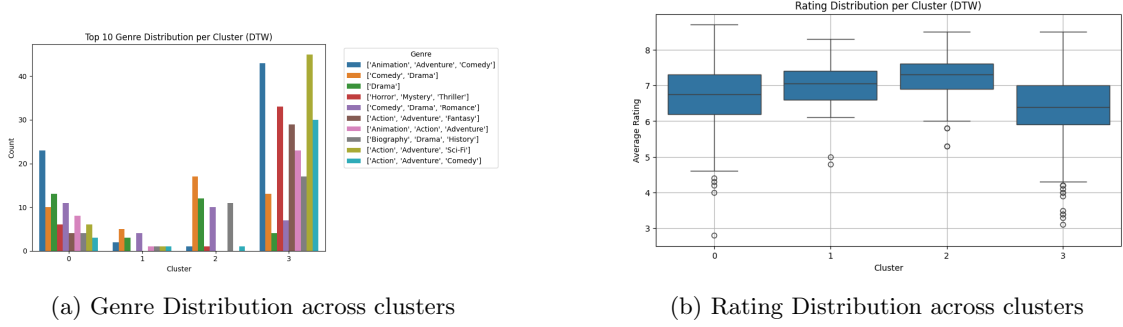


Figure 2.1: Clustering key feature visualization

The plot in figure 2.2 shows the revenue trajectories for the four clusters obtained using KMeans with DTW distance on PAA-reduced time series. Each line represents a time series with the bold one being the centroid of the cluster.

- Cluster 0: Starts high, drops sharply, and slightly recovers toward the end — likely represents movies with strong opening weekends followed by a steep decline and minor long-tail.
- Cluster 1: Relatively flat trend with minor fluctuations — possibly underperforming movies or those with steady but unspectacular earnings.
- Cluster 2: Begins moderately, rises steadily, peaks mid-way, then drops — indicative of movies that gained traction post-release (e.g., word-of-mouth growth).
- Cluster 3: Peaks early and then steadily declines — a typical blockbuster pattern with an initial surge and gradual fade.

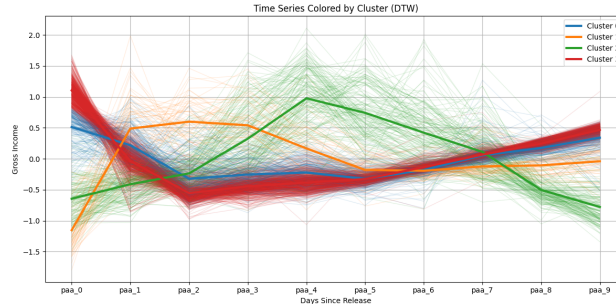


Figure 2.2: Time series colored by clusters

The exploratory genre-cluster correlation further reinforces the idea that genre might influence temporal revenue behavior. This finding opens opportunities for using genre as a prior in hybrid clustering or classification models.

2.2 Motifs and Discords Discovery

2.2.1 Selected Time Series

To obtain more insightful results in the Motifs and Discords analysis, the focus was set on a subset of representative time series. As described in Chapter 1, pre-processed time series were used to ensure comparability

across genres. Firstly, to determine the most appropriate window size, it was computed the matrix profile using different values of $m \in \{3, 5, 7, 10, 14\}$. The corresponding results are shown in Figure 2.3.

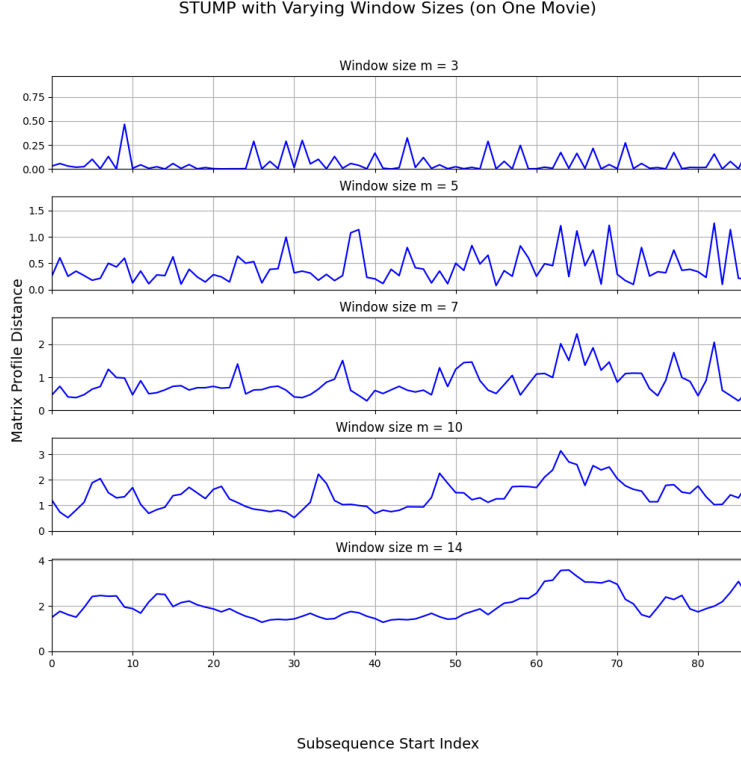


Figure 2.3: Matrix profiles computed with varying window sizes.

This experiment demonstrated how the choice of window size influences the identification of motifs and discords, as illustrated in the corresponding matrix profile plots. The subset included the four centroids from the previous clustering phase. For each centroid series, the window size was set to $m=7$ and the top-3 motifs were extracted using the matrix profile. Motifs were visualized as repeated subsequences within the series, while discords were identified as the subsequences with the highest matrix profile distance values. The resulting plots highlight both the recurring patterns and the anomalous behaviors specific to each centroid, providing a richer understanding of the structural differences across genres.

2.2.2 Motifs & Discords

The first step in performing motif discovery was the computation of the matrix profile of the time series. A crucial aspect of this process concerned the choice of the window size. Different values were tested, and a window size of $w = 7$ was finally adopted, as it provided a reasonable balance between capturing meaningful patterns and ensuring computational efficiency.

For each time series, the top-3 motif groups were extracted. Within each group, the most representative subsequence was designated as the leader (solid line with circular marker), while additional occurrences were displayed as dashed lines with cross markers. This distinction highlights which local shapes are consistently reproduced throughout the signal. In practice, motifs typically correspond to oscillatory behaviours or repeated drops and rises, which represent the stable building blocks of the series.

Discords correspond to subsequences that are least similar to the remainder of the series, identified as the top-3 highest distances in the profile. They were represented as dotted red segments with diamond markers.

These subsequences capture irregularities such as isolated peaks, plateaus, or sudden deviations from the dominant dynamics.

Implementation

The analysis was performed using the stumpy Python library:

- stumpy was applied to compute the matrix profile,
- motifs was used to extract the top-3 motif groups,
- discords were identified as the subsequences corresponding to the three largest profile values.

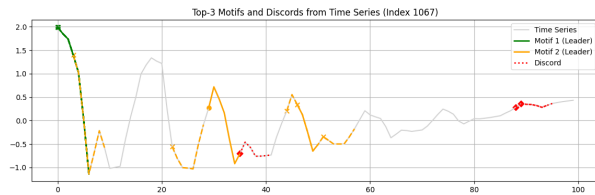
Motif leaders and matches were explicitly distinguished in the visualization to enhance interpretability.

Insights

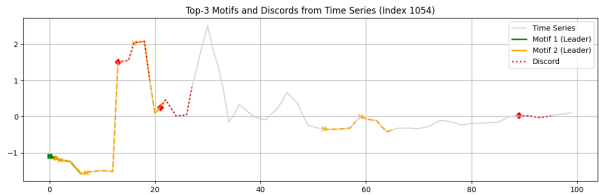
This dual representation provides a compact summary of each time series:

- motifs reveal the regular and recurrent structure,
- discords expose the anomalies and rare behaviours.

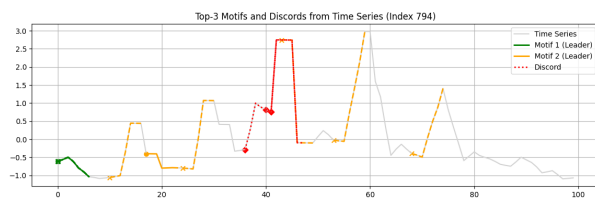
As shown in Figure 2.4, motifs highlight the repeated subsequences in different signals, whereas discords identify the rarest behaviours. For instance, recurrent drops are evident in Index 1067 (Figure 2.4a), oscillatory motifs are retrieved for Index 794 (Figure 2.4b), repeated fluctuations are captured in Index 950 (Figure 2.4c), and structural patterns with rare deviations are observed in Index 1201 (Figure 2.4d).



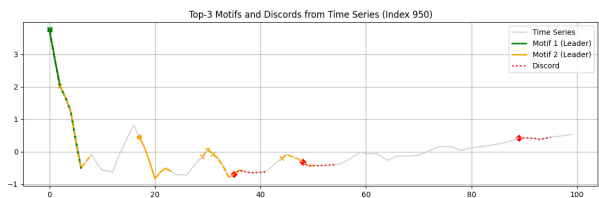
(a) Index 1067: recurrent drops and a discord marking an isolated deviation.



(b) Index 794: oscillatory motifs and discords capturing irregular subsequences.



(c) Index 950: motifs highlight repeating fluctuations, discords isolate spikes.



(d) Index 1201: motifs capture structural patterns, discords reveal rare behaviour.

Figure 2.4: Top-3 motifs (leaders and matches) and discords for four representative time series. Motifs (solid/dashed colored lines with markers) highlight recurrent subsequences, while discords (dotted red with diamonds) expose local anomalies.

2.3 Classification

In this section are described various experiments conducted looking for the best results during a multiclass classification task.

The multiclass classification task was conducted on the approximated and scaled dataset, with the target variable being the "rating_category" of each time series. The original ordinal target had five categories. To reduce sparsity in the lower tail, the categories Medium Low and Low were merged into a single Low class prior to modeling. The resulting four-class distribution was: Medium $n = 387$, High $n = 377$, Medium High $n = 232$, and Low $n = 138$. Class labels were then integer-encoded using LabelEncoder.

A stratified hold-out split was adopted with 20% for testing (test_size=0).

2.3.1 KNeighborsTimeSeriesClassifier

The baseline classifier is KNeighborsTimeSeriesClassifier with $k = 3$ and the DTW distance (distance="dtw"). Table 2.1 reports per-class precision, recall, F1, and support on the test set ($n=227$). Global accuracy and macro- and weighted-averaged scores are also shown.

Table 2.1: k NN (DTW, $k=3$) test performance.

Class (code)	Precision	Recall	F1-score	Support
High (0)	0.42	0.60	0.49	75
Low (1)	0.29	0.32	0.31	28
Medium (2)	0.40	0.33	0.36	78
Medium High (3)	0.22	0.11	0.14	46
Accuracy			0.37	227
Macro avg	0.33	0.34	0.33	227
Weighted avg	0.35	0.37	0.35	227

The DTW baseline attains 37% accuracy and a macro-F1 of 0.33, indicating limited balanced performance across classes. Per-class results suggest uneven behavior: recall is comparatively higher for High (0.60) and lower for Medium High (0.11), consistent with moderate class imbalance and potential confusion between adjacent ordinal categories. Given the PAA compression, it is plausible that short discriminative patterns have been smoothed out, reducing the benefit of elastic matching.

2.3.2 1D Convolutional Neural Network

A compact 1D CNN was trained on univariate inputs of length $T^* = 10$ (PAA compressed), passed as $(N, T^*, 1)$ to sktime's CNNClassifier. The network comprises two convolutional blocks (kernel size 3) with average pooling, followed by flattening and a dense softmax layer. Total parameters: 304. Training used $n_epochs = 100$

Table 2.2 reports test metrics ($n=227$). The confusion matrix shows that predictions concentrate on classes $\{0, 2\}$, with no predictions for classes $\{1, 3\}$.

Table 2.2: CNN (2 conv layers, kernel size 3) — test performance.

Class (code)	Precision	Recall	F1-score	Support
High (0)	0.51	0.55	0.53	75
Low (1)	0.00	0.00	0.00	28
Medium (2)	0.46	0.87	0.60	78
Medium High (3)	0.00	0.00	0.00	46
Accuracy			0.48	227
Macro avg	0.24	0.35	0.28	227
Weighted avg	0.33	0.48	0.38	227

2.3.3 (Mini)ROCKET + Ridge

A MiniROCKET transform with 10,000 kernels was fitted on the training set, producing PPV features; a linear Ridge classifier followed.

Ridge regularization $\alpha \in (-3, 3, 7)$ was selected via internal CV (RidgeClassifierCV). Test metrics:

Table 2.3: MiniROCKET + Ridge (CV) — test performance.

Class (code)	Precision	Recall	F1-score	Support
High (0)	0.54	0.61	0.57	75
Low (1)	0.00	0.00	0.00	28
Medium (2)	0.47	0.81	0.59	78
Medium High (3)	0.43	0.07	0.11	46
Accuracy			0.49	227
Macro avg	0.36	0.37	0.32	227
Weighted avg	0.43	0.49	0.42	227

To mitigate imbalance, class weights were computed on the training set (`compute_class_weight("balanced")`) and used as sample weights in RidgeClassifier. Accuracy deteriorates, but recall and F1 for minority classes increase (Table 2.4).

Table 2.4: MiniROCKET + Ridge (class-weighted) — test performance.

Class (code)	Precision	Recall	F1-score	Support
High (0)	0.41	0.47	0.43	75
Low (1)	0.43	0.36	0.39	28
Medium (2)	0.33	0.31	0.32	78
Medium High (3)	0.22	0.22	0.22	46
Accuracy			0.35	227
Macro avg	0.35	0.34	0.34	227
Weighted avg	0.35	0.35	0.35	227

Table 2.5 juxtaposes the tested models (including the DTW baseline). The best accuracy is obtained by MiniROCKET+Ridge (0.49), while the best macro-F1 among the three learning-based variants is achieved by the class-weighted MiniROCKET (0.34), trading overall accuracy for minority-class recovery.

Table 2.5: Summary of test performance across methods.

Model	Accuracy	Macro-F1
k NN (DTW, $k=3$)	0.37	0.33
CNN (2 conv layers, PAA $T^*=10$)	0.48	0.28
MiniROCKET + Ridge (CV)	0.49	0.32
MiniROCKET + Ridge (class-weighted)	0.35	0.34

All methods exhibit strong confusion between adjacent ordinal categories. Both CNN and MiniROCKET tend to collapse predictions onto High,Medium, with near-zero recall for Low and Medium High unless class weighting is applied. Two factors likely contribute: (i) the aggressive PAA compression that may blur short discriminative motifs; and (ii) the lack of an explicit ordinal inductive bias in the loss or decision rule.

2.4 Shapelet Transform + Random Forest

Shapelets are short, discriminative subsequences. The Shapelet Transform maps each series to the vector of distances to a learned set $\mathcal{S} = \{\mathbf{s}_m\}_{m=1}^M$; a standard tabular classifier is then trained on this feature space. The sktime ShapeletTransformClassifier was used with a RandomForestClassifier backend ($n_estimators = 100$). Inputs were the same univariate PAA-compressed series used elsewhere. Training on the present split produced $M = 807$ shapelets.

The learned dictionary exhibits a marked class skew: 646 shapelets are associated with class code 3 and 161 with class code 2, while none were attached to codes 0 or 1.

Figure 2.5 shows five top-scoring shapelets (all annotated to class 3).

Table 2.6: Counts of discovered shapelets per class code.

Class code	0	1	2	3
# Shapelets	0	0	161	646

Test performance ($n=227$) improves over distance- and feature-kernel baselines, both in accuracy and in macro-averaged F1 (Table 2.7).

Table 2.7: Shapelet Transform + Random Forest — test performance.

Class (code)	Precision	Recall	F1-score	Support
High (0)	0.54	0.59	0.56	75
Low (1)	0.78	0.25	0.38	28
Medium (2)	0.51	0.78	0.62	78
Medium High (3)	0.25	0.09	0.13	46
Accuracy			0.51	227
Macro avg	0.52	0.43	0.42	227
Weighted avg	0.50	0.51	0.47	227

The transform yields the best overall accuracy (0.51) and the highest macro-F1 (0.42) among tested models. Gains stem primarily from improved recall on Medium and High, while Low remains recall-poor despite high precision, and Medium High underperforms (recall 0.09). The concentration of discovered shapelets for

codes 2,3 coupled with poor generalization on code 3 suggests overfitting of local motifs for Medium High and/or insufficient coverage for Low

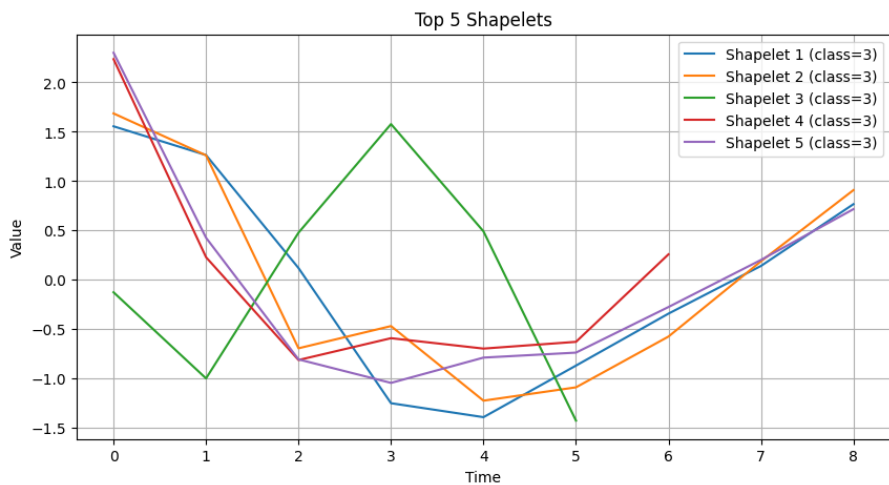


Figure 2.5: Top five learned shapelets (all associated with class code 3).

3 Advanced Pre-Processing

3.1 Outlier Detection

The IMDB dataset was analyzed to detect anomalous records whose metadata significantly deviates from the majority of movies. Identifying outliers is critical both for improving data quality and for discovering exceptional cases with unusual commercial or critical performance.

To ensure a robust evaluation, four complementary families of unsupervised outlier detection techniques were applied: distance-based, angle-based, density-based, and ensemble-based (Isolation Forest). These methods exploit different assumptions about what constitutes an anomaly, enabling both global and local irregularities to be identified.

3.1.1 Methodology

1. Distance-Based Outlier Detection

This approach assumes that anomalous observations are far from the majority of data points in the feature space. For each sample, it was computed the distance to its k nearest neighbors using the Euclidean metric. Points whose average distance to neighbors exceeded a threshold (e.g., the 95th percentile of the distance distribution) were flagged as potential outliers. In practice, it is controlled the expected proportion of anomalies by setting the contamination parameter to 0.01, meaning that the algorithm assumes approximately 1% of the data are outliers. This method is sensitive to scale but effective at highlighting global anomalies in continuous numerical features. The KNN approach classified 815 points as outliers.

2. Angle-Based Outlier Detection (ABOD)

Distance-based techniques can fail in high-dimensional spaces where distances become less informative. To address this, it was applied the Angle-Based Outlier Detection (ABOD) algorithm, which identifies anomalies based on the variance of angles formed between data points and their neighbors. Anomalies tend to exhibit smaller angular variance because they are isolated in sparse regions. ABOD is particularly suitable for detecting subtle outliers in high-dimensional datasets. It was observed that setting the number of neighbors to 50 produced the best results for the structure of our dataset. Consequently, the model was configured with the following parameters: `K=50`, `contamination=0.01`, `distance='euclidean'`, and `methd='fast'`. The ABOD approach detected 1159 points as outliers.

3. Density-Based Approaches

The Local Outlier Factor (LOF) algorithm compares the local density of a point to that of its neighbors. Points in low-density regions relative to their surroundings are flagged as outliers, making LOF effective at detecting context-dependent anomalies. For example, moderately popular movies in a cluster of highly popular ones. The model was configured with the following parameters: `algorithm='auto'`, `contamination=0.01`, `leaf_size=30`, `metric='minkowski'`, `n_neighbors=10`. The LOF approach detected 1166 points as outliers.

4. Isolation Forest

Finally, it was applied the Isolation Forest, an ensemble-based algorithm designed for multivariate anomaly detection in high-dimensional spaces. The method constructs multiple binary trees by randomly selecting split thresholds across features. Points that require fewer splits to isolate are considered outliers. This algorithm detected 1166 points as outliers.

3.1.2 Dimensionality Reduction and Visualization

To visualize the structure of the dataset and better interpret outlier behavior, Principal Component Analysis (T-SNE) and Uniform Manifold Approximation and Projection (UMAP), to project the numerical feature space into two dimensions, were applied.

3.1.3 Outliers Management

In the final stage of the analysis, the adopted approaches were compared. Outliers were assigned a severity level based on the level of consensus among the methods:

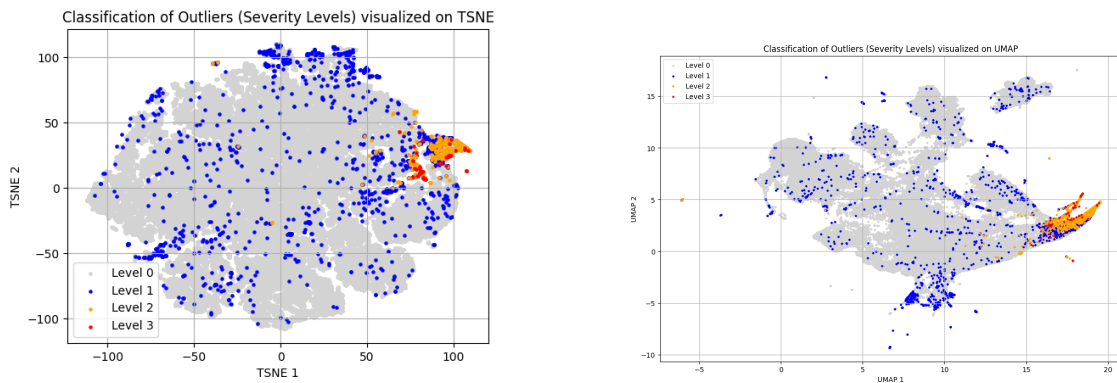
1. Level 3 – Hard Outliers: flagged by all four methods.
2. Level 2 – Moderate Outliers: detected by two or three methods simultaneously.
3. Level 1 – Weak Outliers: identified by only one method.
4. Level 0 – Normal Points: consistently considered inliers.

After assigning severity levels, all normal points (Level 0) and weak outliers (Level 1) were retained while discarding moderate and hard anomalies. This strategy preserves data diversity while removing extreme outliers that could distort downstream analyses.

The cleaned dataset contained 115 753 instances out of the original 116 539, ensuring a balanced trade-off between quality and representativeness.

To visualize this, it was projected the high-dimensional feature space into 2D using UMAP and TSNE with color-coded points based on their outlier severity (Figure 3.1). This representation highlights a clear separation between normal observations and anomalies, where hard outliers (red) are well-isolated from the dense central region. It was used a simple intersection method for the obtained results. This strategy allowed to identify anomalies shared across the different methods.

To visually validate the impact of the outlier detection and cleaning process, it was compared the distributions of the numerical features before and after processing using boxplots (Figure 3.2). In the top panel, which represents the raw dataset, several numerical variables show extreme outliers, leading to highly skewed distributions and compressed interquartile ranges. These extreme values dominate the scale and obscure meaningful variance among the majority of observations. After removing moderate and hard outliers (Level 2 and Level 3 anomalies), as shown in the bottom panel, the distributions become significantly more compact and interpretable.



(a) 2D TSNE Representation of dataset and outliers

(b) 2D UMAP Representation of dataset and outliers

Figure 3.1: 3D PCA Repr. of the original dataset with outliers identified by red points.

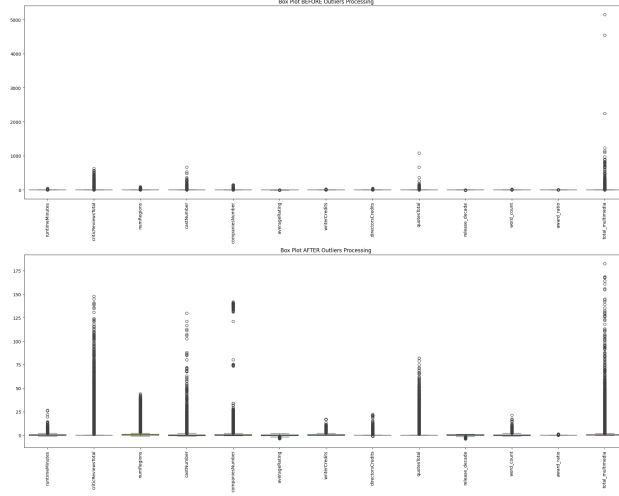


Figure 3.2: Comparison of feature distributions before and after outlier processing. Top: raw dataset; Bottom: cleaned dataset

3.2 Imbalanced Learning

In this section, a predictive analysis is presented using basic classifiers such as the Decision Tree (DT) and K-Nearest Neighbors (KNN). The goal was to predict the binary, imbalanced attribute `is_adult`, which takes the value 1 when the movie is for adults only and 0 when it is not. The dataset was split into 80% for training and 20% for testing.

The target attribute explicit was selected due to its imbalance, as required by the task:

- 98% of the samples are labeled as 0.
- 12% of the samples are labeled as 1.

For both base models, an initial parameter tuning was performed using a randomized search, which returned the following optimal values:

- Decision Tree: `min_samples_split = 40`, `min_samples_leaf = 20`, `max_depth = 50`, `criterion = 'entropy'`
- KNN: `n_neighbors = 11`, `p = 1`, `weights = 'distance'`

The results of this analysis are shown in Table 3.1.

Model	Class	Accuracy	Precision	Recall	F1-Score	Support
Decision Tree	0	1.00	1.00	1.00	1.00	22664
	1		0.94	0.81	0.87	438
KNN	0	1.00	1.00	1.00	1.00	22664
	1		0.82	0.77	0.79	438

Table 3.1: Performance metrics for Decision Tree and KNN models.

The results obtained from the classifiers, shown in Table 3.1, are very good, with both accuracy scores of 1.00, respectively. However, when observing other metrics such as F1score and Recall, there is room

for improvement for the minority class. This can be attributed to the class imbalance in the dataset, as highlighted at the beginning of the section.

By applying specific balancing techniques to the training set, it is possible to address this imbalance and aim for better results. To further enhance performance, it is decided to perform parameter tuning for the classifiers each time a new balancing technique is applied.

3.2.1 UnderSampling

The core idea is to remove samples from the majority class to achieve a more balanced distribution between the two classes. The techniques used are Random Undersampler, Edited Nearest Neighbors (ENN), and Tomek Links.

The first technique applied was the Random Undersampler, which randomly reduces the number of samples in the majority class to match that of the minority class. Using this method, the number of samples in class 0 was reduced to 1750.

The second technique used was Tomek Links, which identifies and removes specific samples that are likely noisy or located near the decision boundary between classes. This improves the separation between the classes. After applying this technique, the number of samples in class 0 was reduced to 90489.

Finally, it was adopted the Edited Nearest Neighbors (ENN) technique, which removes samples that do not match the majority of their Knearest neighbors. The application of ENN reduced the number of samples in class 0 to 89124.

All results obtained were analyzed and compared to evaluate the impact of each balancing technique on the classifier's performance (Table:3.2)

Balancing Method	Model	Class	Accuracy	Precision	Recall	F1-Score
Random Undersampling	Decision Tree	0	0.96	1.00	0.96	0.98
		1		0.33	0.94	0.49
	KNN	0	0.88	1.00	0.88	0.94
		1		0.14	0.97	0.24
Tomek Links	Decision Tree	0	1.00	1.00	1.00	1.00
		1		0.93	0.82	0.87
	KNN	0	0.92	1.00	1.00	1.00
		1		0.81	0.78	0.79
ENN	Decision Tree	0	0.99	1.00	1.00	1.00
		1		0.88	0.85	0.86
	KNN	0	0.99	1.00	0.99	0.99
		1		0.61	0.80	0.69

Table 3.2: Performance metrics for Decision Tree and KNN with undersampling techniques (Random Undersampling, Tomek Links, ENN).

As expected, the Random Undersampler performed the worst due to its random selection of samples, which can lead to the loss of valuable information. On the other hand, the other techniques—Tomek Links and Edited Nearest Neighbors (ENN)—slightly improved both precision and recall compared to the original model.

3.2.2 OverSampling

The idea behind oversampling techniques is to add samples to the minority class. Using the Random Oversampler, which increases the number of samples randomly, the number of samples in class 1 was increased

to 90655, equal to that of the majority class. The second technique used was SMOTE (Synthetic Minority Over-sampling Technique), which creates synthetic samples through interpolation between neighboring minority class samples. With this technique, the number of samples in class 1 was also increased to 90655. Finally, it was applied the ADASYN (Adaptive Synthetic Sampling) technique. Unlike SMOTE, ADASYN generates new samples adaptively, with greater focus on areas where the minority class is underrepresented and harder to learn. Using this approach, the number of samples in class 1 reached 90552. Similarly, when evaluating the overall performance of the classification models on the balanced dataset, a slight decrease in accuracy can be observed compared to the corresponding values derived from the imbalanced dataset. However, in contrast, class 1 shows an increase or substantial improvement in recall (Table:3.3).

Balancing Method	Model	Class	Accuracy	Precision	Recall	F1-Score
Random	Decision Tree	0	0.99	1.00	1.00	1.00
		1		0.85	0.81	0.83
	KNN	0	0.99	0.99	0.99	0.99
		1		0.70	0.71	0.71
SMOTE	Decision Tree	0	0.99	1.00	0.99	0.99
		1		0.86	0.82	0.84
	KNN	0	0.98	0.95	0.87	0.91
		1		0.56	0.79	0.66
ADASYN	Decision Tree	0	0.81	1.00	0.99	1.00
		1		0.76	0.85	0.80
	KNN	0	0.83	1.00	0.99	0.99
		1		0.55	0.79	0.65

Table 3.3: Performance metrics for Decision Tree and KNN with oversampling techniques (Random, SMOTE, ADASYN).

3.2.3 Conclusions

In conclusion, it can be stated that while the overall ability of the models to correctly predict instances has been slightly reduced, the capacity to correctly identify positive examples has significantly increased. The best performing algorithm was Decision Tree with ENN, by removing majority-class samples whose labels conflicted with the majority of their k nearest neighbors (k=3), it effectively eliminated ambiguous non-adult points located near adult clusters. This selective cleaning improved the local separability of minority regions without excessively reducing the dataset size. As a result, ENN enhanced classification precision while preserving most of the informative majority examples, achieving a better balance between precision to other balancing techniques.

4 Advanced Machine Learning

This chapter presents the results obtained through the development of classification and regression tasks. The data underwent preprocessing steps presented in previous chapters: including feature selection, standardization to ensure consistent scaling, and the removal of detected outliers. For both tasks, categorical variables were transformed using appropriate encoding techniques.

4.1 Advanced Classification

Various techniques were used for classifying the target variable "rating_category". The algorithms employed include Neural Networks, Ensemble Methods, and Gradient Boosting Machines.

The target variable, rating_category, was engineered from the continuous variable averageRating by discretizing it into five ordinal classes using quantile-based binning (pd.qcut). This ensures an approximately equal distribution of samples across categories, avoiding severe imbalance. This transformation frames the modeling task as a 5-class classification problem, where the model predicts the qualitative rating band of a movie.

4.1.1 Baseline Model: Perceptron

Before developing more complex neural network architectures, it was established a baseline using the Perceptron algorithm. Despite its simplicity, the Perceptron serves as a valuable reference point for evaluating the performance gains achieved by more advanced models.

The model was trained using a 5-fold Stratified Cross-Validation strategy to ensure balanced representation of all five rating categories across folds. The scikit-learn implementation of Perceptron was used with default hyperparameters and a fixed random seed for reproducibility.

Table 4.1 presents the performance of the Perceptron across the five rating categories obtained through cross-validation.

Table 4.1: Perceptron Classification Report (5-Fold Cross-Validation)

Class	Precision	Recall	F1-Score	Support
0 (Very Low)	0.25	0.22	0.23	25,541
1 (Low)	0.24	0.28	0.26	23,740
2 (Medium)	0.20	0.26	0.23	22,550
3 (High)	0.25	0.29	0.27	19,961
4 (Very High)	0.35	0.20	0.25	23,715
Accuracy		0.25		
Macro Avg	0.26	0.25	0.25	115,507
Weighted Avg	0.26	0.25	0.25	115,507

The Perceptron achieved an overall accuracy of approximately 25%, which is slightly better than random guessing for a five-class problem (20%). However, the performance varies significantly across categories: while class 4 achieves the highest precision (35%), its recall is relatively low, suggesting underrepresentation in predictions. Conversely, class 3 shows balanced recall and precision but moderate F1-score. Figure 4.1 shows ROC curve of trained perceptron.

These results highlight that a simple linear decision boundary is insufficient to capture the complex relationships among features.

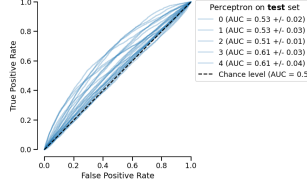


Figure 4.1: ROC curve of Perceptron Baseline Model

Consequently, the Perceptron serves as a baseline benchmark, justifying the exploration of more advanced architectures such as feed-forward neural networks and ensemble methods.

4.2 Multi-Layer Perceptron (MLP)

After establishing the Perceptron as a baseline, it was designed and trained a more expressive Multi-Layer Perceptron (MLP) to improve classification performance on the movie dataset.

4.2.1 Model Architecture

The implemented MLP uses a feed-forward architecture designed to capture nonlinear relationships between features. Firstly it was trained a simple MLP with Input Layer matching the dimensionality of the processed feature space ($X.shape[1]$), one fully-connected layer with 120 neurons and ReLU activation and an output softmax layer with 5 units, producing class probabilities for the rating categories.

The model was compiled using the Adam optimizer ($\eta = 0.001$) and the sparse categorical cross-entropy loss function, which is well-suited for multi-class classification tasks with integer-encoded labels.

To ensure robust performance estimation, it was adopted a stratified 5-fold cross-validation strategy, preserving the class distribution in each fold.

For each fold, the model was trained on 80% of the data and validated on the remaining 20%. Performance metrics were averaged across folds.

Figure 4.2 shows the evolution of training and validation accuracy across epochs, averaged over the five folds.

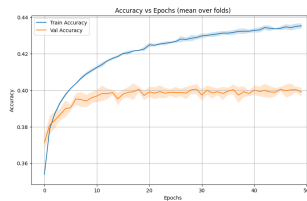


Figure 4.2: Training vs Validation Accuracy (mean over folds).

Key observations:

- The model exhibits rapid accuracy gains during the first 10 epochs, after which improvements slow down.
- The validation accuracy plateaus around $\sim 40\%$, which significantly outperforms the Perceptron baseline ($\sim 25\%$).
- A persistent generalization gap is visible, indicating overfitting.

Using cross-validation predictions aggregated over all folds, the MLP achieved:

- Overall accuracy: 39% (+15% compared to the baseline).
- Macro-averaged F1-score: ≈ 0.38 .
- Substantial improvements in the Medium and High rating categories, demonstrating the model’s ability to capture nonlinear feature interactions.

Discussion and Insights

The MLP significantly outperforms the Perceptron, highlighting the importance of nonlinear modeling for this dataset. However, the following challenges remain:

1. The moderate performance ($\sim 40\%$ accuracy) suggests high feature overlap between classes, making separation difficult.
2. The observed generalization gap slight overfitting.

This MLP serves as a strong intermediate baseline for the multi-class classification problem, bridging the gap between simple linear models and more complex ensemble methods.

Class	Precision	Recall	F1-Score	Support
0 (Very Low)	0.37	0.44	0.40	25,541
1 (Low)	0.36	0.26	0.30	23,740
2 (Medium)	0.30	0.20	0.24	22,550
3 (High)	0.37	0.44	0.40	19,961
4 (Very High)	0.49	0.60	0.54	23,715
Accuracy			0.39	115,507
Macro Avg	0.38	0.39	0.38	115,507
Weighted Avg	0.38	0.39	0.38	115,507

Table 4.2: Classification report for MLP using cross-validation predictions.

4.2.2 Hyperparameter Optimization and Final MLP Model

To improve upon the baseline MLP, a comprehensive hyperparameter optimization strategy was employed using KerasTuner with a custom cross-validation tuner (CVTuner). This approach integrates a stratified 3-fold cross-validation within the hyperparameter search, ensuring a more robust estimate of generalization performance.

Search Space and Strategy

The tuning process explored a rich search space, including:

- Number of hidden layers: $\{1, 2, 3\}$.
- Neurons per layer: $\{64, 128, 256\}$.
- Dropout rate per layer: 0.0 to 0.5 (step 0.1).
- Regularization type: none, ℓ_1 , or ℓ_2 , with continuous weight search.
- Batch Normalization: enabled/disabled per layer.
- Optimizer: SGD (tuning momentum and learning rate) or Adam (tuning β_1 and learning rate).

The tuning was performed over 10 trials with 50 epochs each, leveraging early stopping (*patience* = 5) to avoid overfitting.

Best Hyperparameters

The optimal configuration identified by the tuner was:

- **Number of layers:** 3.
- **Layer 1:** 256 units, dropout 0.3, batch normalization enabled, no regularization.
- **Layer 2:** 128 units, dropout 0.4, batch normalization disabled, ℓ_2 regularization applied.
- **Layer 3:** 128 units, dropout 0.0, batch normalization enabled, no regularization.
- **Optimizer:** Adam with $\beta_1 = 0.95$ and learning rate $\eta = 3.95 \times 10^{-5}$.

This configuration achieved a mean cross-validated validation accuracy of $\approx 40.5\%$, marking a substantial improvement over both the baseline Perceptron ($\approx 25\%$) and the simple MLP ($\approx 39\%$).

Final Evaluation on Test Set

Using the optimal hyperparameters, the final model was retrained on the entire training set (80% of the data), reserving 20% for internal validation. Early stopping was again employed to avoid overfitting. The optimized MLP was evaluated on a held-out test set ($n = 23,074$), yielding the following results:

Table 4.3: Final MLP Performance on Test Set

Class	Precision	Recall	F1-Score	Support
0 (Very Low)	0.37	0.47	0.42	5,104
1 (Low)	0.38	0.31	0.34	4,743
2 (Medium)	0.36	0.19	0.25	4,565
3 (High)	0.39	0.46	0.42	3,982
4 (Very High)	0.57	0.63	0.60	4,740
Accuracy			0.41	
Macro Avg	0.40	0.41	0.40	23,074
Weighted Avg	0.40	0.41	0.40	23,074

Figure 4.3 shows a significant improvement of the model performance with respect to the simple initially trained Perceptron.

Insights and Interpretation

Key findings from this analysis include:

- The optimized MLP demonstrates a significant improvement over both the Perceptron and the simple MLP, achieving $\sim 41\%$ test accuracy.
- Performance across classes is uneven: while the model performs well on extreme ratings (classes 0 and 4), intermediate ratings (especially class 2) remain challenging.
- Batch normalization and dropout proved essential for stabilizing training and reducing overfitting.
- The carefully tuned Adam optimizer, combined with low learning rates, contributed to achieving stable convergence.

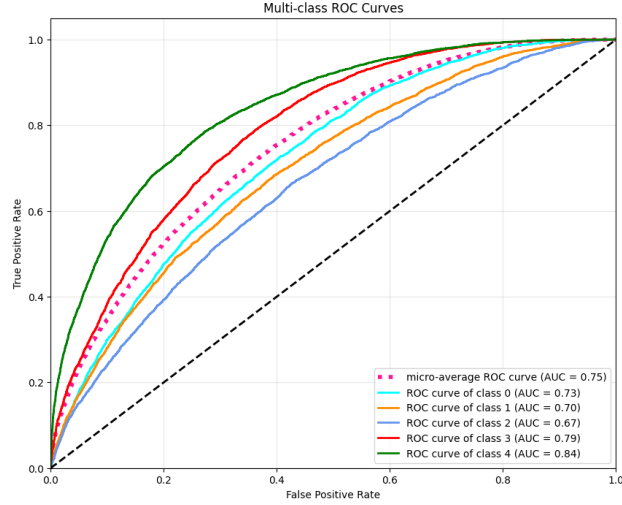


Figure 4.3: Roc Curve of Fine Tuned MLP

4.3 Ensemble methods

Ensemble Methods combine the predictions of multiple base estimators to improve generalization and robustness compared to using a single estimator. The two main approaches in ensemble methods are bagging and boosting:

- In bagging, multiple classifiers are trained independently on randomly sampled subsets of the training set. The final predictions are obtained by aggregating the predictions of all classifiers, often using majority voting or averaging.
- In boosting, "weak learners" are trained sequentially. Each learner attempts to correct the errors of the previous model by assigning higher weights to the data points that were misclassified. These misclassified data points are more likely to be selected in subsequent models, allowing the ensemble to focus on the more challenging cases.

In this section, the following methods will be explored: - Random Forest (a bagging-based method), - AdaBoost, and - Gradient Boosting Machines.

4.3.1 Random Forest

The first method applied was Random Forest, which uses the bagging technique to combine the results of multiple decision trees (usually hundreds or thousands), creating a strong model that is particularly effective for both classification and regression tasks. The hyperparameters were tuned using a RandomizedSearchCV strategy with a 5-fold cross-validation and 10 iterations, optimizing across the following search space:

`n_estimators` $\in \{50, 100, 150, 200\}$, `min_samples_split` $\in \{2, 5, 10, 20\}$, `min_samples_leaf` $\in \{1, 5, 10, 20\}$, `max_depth` $\in [2, 10]$, `criterion` $\in \{\text{gini}, \text{entropy}, \text{log_loss}\}$, `max_features` $\in \{\text{sqrt}, \text{log2}\}$

The optimal hyperparameters found were: `n_estimators` = 50, `max_depth` = 8, `min_samples_split` = 20, `min_samples_leaf` = 5, `max_features` = log2, `criterion` = gini.

With the complete set of features, the model achieved the following results:

- Training accuracy: 36.70%
- Test accuracy: 35.55%

- Weighted F1-score: 0.28

The classification report indicated that the model struggled particularly on the “Medium” and “Very High” classes, achieving recall values as low as 0.04 and 0.18, respectively.

To better understand model behavior, feature importances were extracted using the Mean Decrease in Impurity (MDI). Figure 4.4 highlights the ten most influential features, with `titleType_tvEpisode`, `runtimeMinutes`, and `titleType_movie` emerging as the strongest predictors.

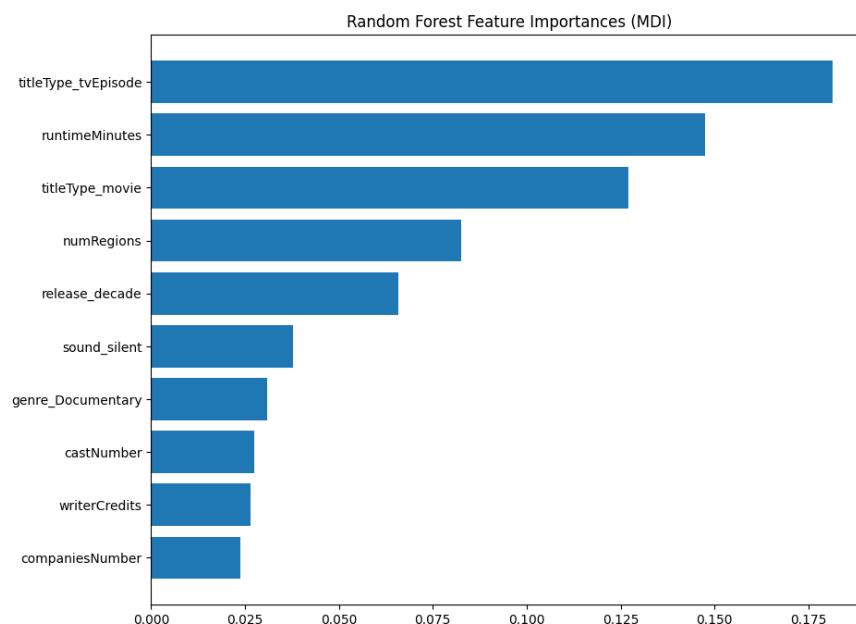


Figure 4.4: Top 10 most important features for the Random Forest model.

Using only the 15 most important features, the model was retrained and evaluated on the reduced dataset. The resulting performance slightly improved:

- Test accuracy: 36.23%
- Weighted F1-score: 0.32

This shows that removing irrelevant or less impactful features helped the model generalize better, while keeping performance comparable to the full model.

Overall, Random Forest performed moderately well but not better than a fine tuned MLP.

4.3.2 Boosting Methods

Boosting is an ensemble learning technique where multiple weak learners are combined sequentially to form a strong classifier. Unlike bagging methods, such as Random Forests, where models are trained independently in parallel, boosting focuses on training models sequentially. Each subsequent model attempts to correct the mistakes made by the previous ones, assigning higher weights to misclassified samples to improve overall performance.

In this section, it was explore three boosting-based methods:

- AdaBoost (Adaptive Boosting),
- XGBoost (Extreme Gradient Boosting),
- LightGBM (Light Gradient Boosting Machine).

AdaBoost

AdaBoost is one of the earliest and most widely used boosting algorithms. It iteratively trains weak learners (usually decision stumps) and assigns higher weights to misclassified examples so that subsequent models focus on harder samples.

A RandomizedSearchCV with 5-fold cross-validation and 10 iterations was used to search over the following parameter grid: `n_estimators` $\in \{50, 75, 100, 125\}$, `learning_rate` $\in \{0.01, 0.1, 0.2, 0.3, 0.5\}$, `algorithm` $\in \{SAMME, SAMME.R\}$.

The best parameters found were: `n_estimators` = 125, `learning_rate` = 0.3, `algorithm` = SAMME.R.

- Training Accuracy: 34.55%
- Test Accuracy: 32.03%
- Weighted F1-score: 0.24

Feature importance analysis shows that `titleType_tvEpisode`, `release_decade`, and `titleType_movie` contributed the most to the model's predictions. Despite its simplicity, AdaBoost struggled to capture the complexity of this dataset, resulting in limited predictive power.

XGBoost

XGBoost, short for Extreme Gradient Boosting, improves upon traditional boosting by implementing gradient boosting techniques and adding significant optimizations such as regularization, parallelization, and efficient handling of missing values. The following hyperparameter grid was explored: `n_estimators` $\in \{50, 75, 150\}$, `learning_rate` $\in \{0.01, 0.05, 0.1\}$, `max_depth` $\in \{3, 5, 7, 9\}$, `subsample` $\in \{0.6, 0.8, 1.0\}$, `colsample_bytree` $\in \{0.6, 0.8, 1.0\}$, `reg_lambda` $\in \{0.0, 0.5, 1.0\}$, `reg_alpha` $\in \{0.0, 0.5, 1.0\}$.

Best configuration: `n_estimators` = 150, `learning_rate` = 0.01, `max_depth` = 9, `subsample` = 0.8, `colsample_bytree` = 1.0, `reg_lambda` = 0.0, `reg_alpha` = 0.0.

Performance Results

- Training Accuracy: 39.50%
- Test Accuracy: 39.50%
- Weighted F1-score: 0.40

Feature importance analysis reveals that `titleType_movie`, sentiment-related features, and genre-based attributes were key drivers of predictive performance. XGBoost achieved the best overall performance, demonstrating its ability to handle complex relationships and imbalanced distributions.

LightGBM

LightGBM, developed by Microsoft, is an efficient implementation of gradient boosting that uses histogram-based techniques and leaf-wise tree growth. It is designed to be computationally efficient while maintaining competitive predictive performance.

The following parameter search space was used: `boosting_type` $\in \{gbdt, goss\}$, `n_estimators` $\in \{50, 100, 200\}$, `learning_rate` $\in \{0.001, 0.01, 0.05, 0.1\}$, `max_depth` $\in \{3, 5, 7, 9\}$, `num_leaves` $\in \{15, 31, 63\}$, `min_data_in_leaf` $\in \{100, 200, 300\}$, `reg_alpha` $\in \{0.0, 0.1, 0.3, 0.5\}$, `reg_lambda` $\in \{0.0, 0.1, 0.3, 0.5\}$.

Best configuration: `boosting_type` = goss, `n_estimators` = 200, `learning_rate` = 0.05, `max_depth` = 7, `num_leaves` = 31, `reg_alpha` = 0.1, `reg_lambda` = 0.0.

- Training Accuracy: 38.7%
- Test Accuracy: 38.7%
- Weighted F1-score: 0.36

LightGBM also produced consistent ROC curves across classes, achieving an AUC range of approximately 0.63 to 0.81 (Figure 4.5).

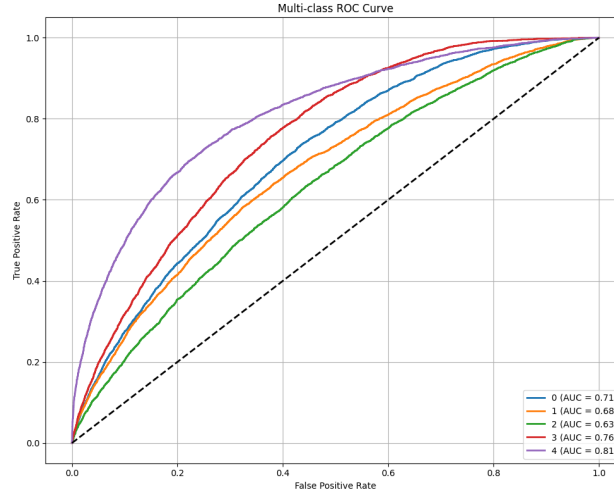


Figure 4.5: Multi-class ROC curve for LightGBM.

Comparison of Boosting Models

Table 4.4 summarizes the performance of the boosting models:

Model	Test Accuracy	Weighted F1	AUC (avg)
AdaBoost	32.03%	0.24	0.60
LightGBM	38.70%	0.36	0.72
XGBoost	39.50%	0.40	0.74

Table 4.4: Comparison of boosting models on the test set.

From the results, XGBoost achieved the highest performance in terms of both accuracy and F1-score, while LightGBM offered competitive performance with lower computational cost. AdaBoost, while conceptually simpler, struggled with the dataset’s imbalance and high-dimensional feature space.

4.4 Advanced Regression

Regression is a fundamental task in supervised learning where the goal is to predict a continuous target variable based on input features. While linear regression is simple and interpretable, it often fails to capture complex non-linear relationships.

In this study, it was use two powerful non-linear models:

- Random Forest Regressor – an ensemble method based on bagging and decision trees.
- GBM Regressor – a gradient boosting framework.

For both models, hyperparameter tuning was performed using RandomizedSearchCV and HalvingRandomSearchCV. A 5-fold cross-validation strategy was applied for consistent evaluation.

4.4.1 Random Forest Regressor

The Random Forest Regressor is an ensemble learning method that trains multiple decision trees and averages their predictions. A grid search was performed testing the following hyperparameters: Number of estimators $\in \{100, 200, 300\}$, Maximum depth $\in \{10, 20, \text{None}\}$, Minimum samples per split $\in \{2, 5, 10\}$, Minimum samples per leaf $\in \{1, 2, 4\}$, Maximum features $\in \{\text{sqrt}, \log 2\}$, with bootstrap sampling enabled.

Performance Metrics

The following metrics were used: R^2 Score, Mean Squared Error (MSE), Mean Absolute Error (MAE)

Table 4.5: Random Forest Performance

	R^2	MSE	MAE
Training Set	0.91	0.05	
Test Set	0.42	0.35	0.43

4.4.2 LightGBM Regressor

LightGBM is a gradient boosting framework optimized for speed and efficiency. It uses a leaf-wise tree growth strategy and supports large-scale datasets. The hyperparameter configuration for the LightGBM model was defined as follows: Boosting types $\in \{\text{gbdt}, \text{goss}, \text{dart}\}$, Learning rate $\in \{0.001, 0.01, 0.05, 0.1\}$, Number of estimators $\in \{50, 100, 200\}$, Maximum depth $\in \{3, 5, 7, 9\}$, Regularization terms λ and α , and objective set to regression.

Table 4.6: LightGBM Performance

	R^2	MSE	MAE
Training Set	0.91	0.05	
Test Set	0.42	0.35	0.43

4.4.3 Results and Comparison

Based on the results obtained from the experiments:

- Random Forest achieved a higher R^2 score on the training set, indicating strong fitting capabilities, but showed slightly higher generalization error on the test set.
- GBM, despite being faster to train, achieved comparable R^2 and lower MSE on the test set, showing better robustness on unseen data.
- Residual plots revealed that LightGBM handled outliers more effectively, while Random Forest residuals were more evenly distributed but slightly larger in magnitude.

This study demonstrated how two advanced regression techniques can be applied to a multiple regression problem. Both models provide competitive results, but the choice between them depends on the trade-off between computational efficiency and predictive accuracy.

5 Explainable AI

While predictive performance is essential, complex machine learning models such as gradient boosting often operate as “black boxes”, making it difficult to understand the reasons behind their decisions. In sensitive domains, this lack of transparency may hinder trust, accountability, and the possibility of model validation. To address this issue, explainable AI (XAI) techniques were applied to the CatBoost classifier in order to gain both a global and a local understanding of its behavior. Global explanations provide an overview of which features drive the predictions across the dataset, while local explanations reveal why the model produced a specific output for an individual instance.

5.0.1 Global Explanations

First, feature importances were computed using CatBoost’s native SHAP implementation. Figure 5.1 shows the top 20 features ranked by their mean absolute SHAP value. The most influential predictors are `total_multimedia`, `release_decade`, `castNumber`, and `runtimeMinutes`. This suggests that structural and temporal characteristics of a movie are the strongest indicators for distinguishing adult content.

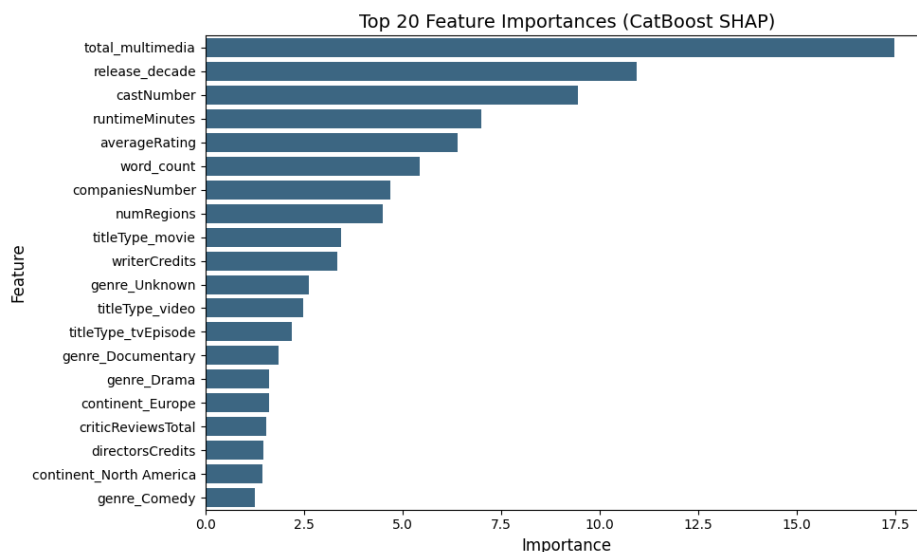


Figure 5.1: Top 20 feature importances based on SHAP values.

To obtain a more detailed view, a SHAP beeswarm plot was employed (Figure 5.2). Each dot represents a sample in the test set, positioned according to the SHAP value of the corresponding feature. The color encodes the actual feature value (red = high, blue = low). From this perspective it is possible to observe that higher values of `total_multimedia` and more recent `release_decade` values increase the probability of being classified as adult, whereas certain genres such as `Documentary` or `Comedy` tend to reduce it. The feature `titleType_video` has a clear positive effect, strongly pushing predictions towards the adult class. This indicates that the model not only learns monotonic trends but also captures categorical factors that influence predictions in a consistent manner.

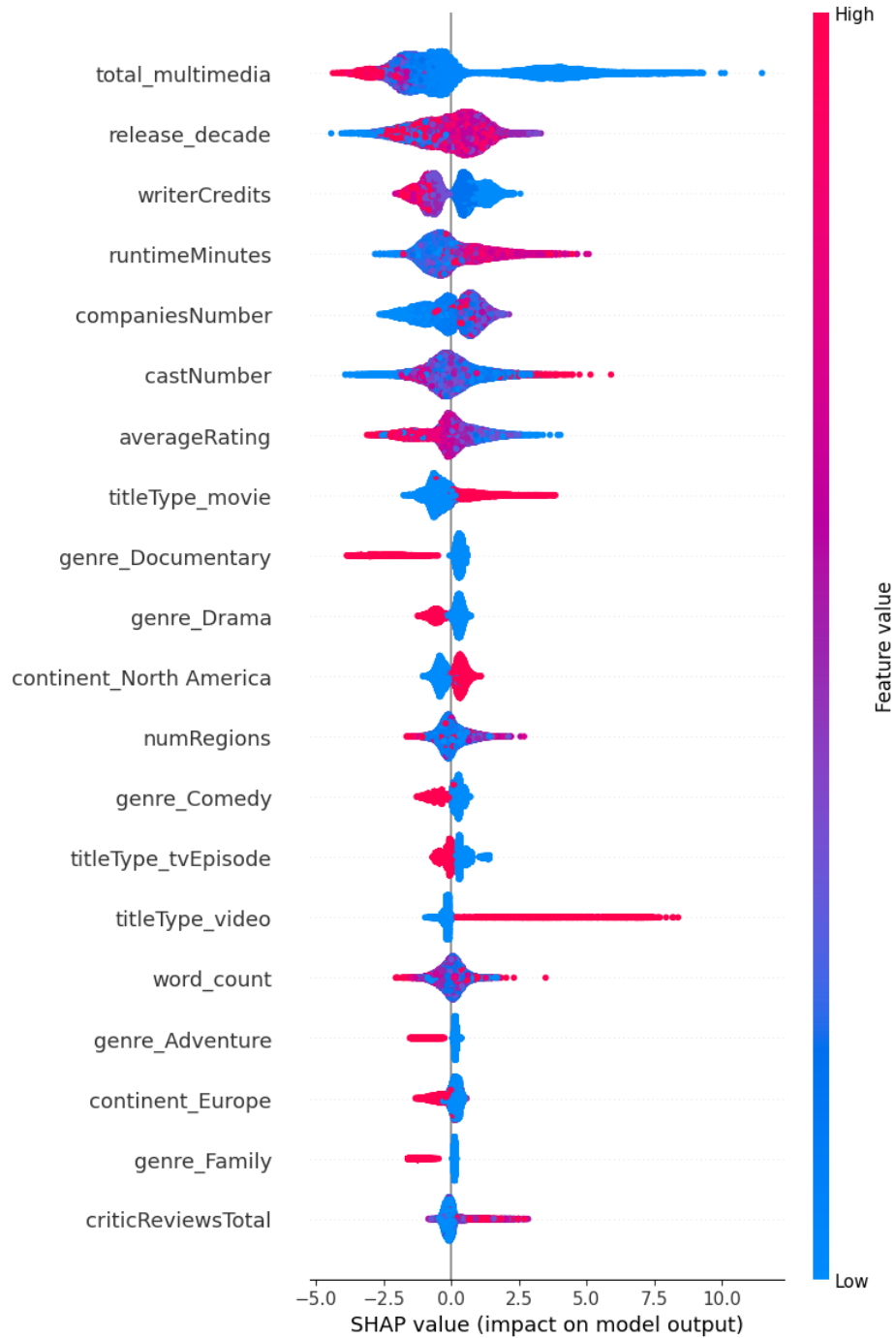


Figure 5.2: SHAP beeswarm plot: global explanation of feature impact.

To further enrich the analysis, feature interactions were investigated by computing CatBoost's interaction importances (Table 5.1). The strongest pairwise effects involve runtimeMinutes with castNumber, as well as combinations of castNumber and averageRating. These results highlight that the model does not rely exclusively on single variables, but also captures meaningful nonlinear interactions that reflect realistic

dependencies among movie characteristics.

Table 5.1: Top 10 feature interactions (CatBoost SHAP).

Feature 1	Feature 2	Interaction Score
runtimeMinutes	castNumber	2.65
castNumber	averageRating	2.19
runtimeMinutes	total_multimedia	2.10
runtimeMinutes	averageRating	2.10
castNumber	total_multimedia	2.00
averageRating	total_multimedia	1.75
release_decade	total_multimedia	1.67
averageRating	word_count	1.34
runtimeMinutes	titleType_video	1.31
runtimeMinutes	word_count	1.19

An example of an interaction dependence plot is shown in Figure 5.3. Here, the SHAP value of runtimeMinutes is plotted against its actual value, with points colored by castNumber. The plot shows that medium-length films with large casts strongly increase the probability of being classified as adult, while very short or very long films have a weaker impact. This visualization provides evidence of a nonlinear effect in which duration interacts with cast size to modify the prediction in a non-trivial way.

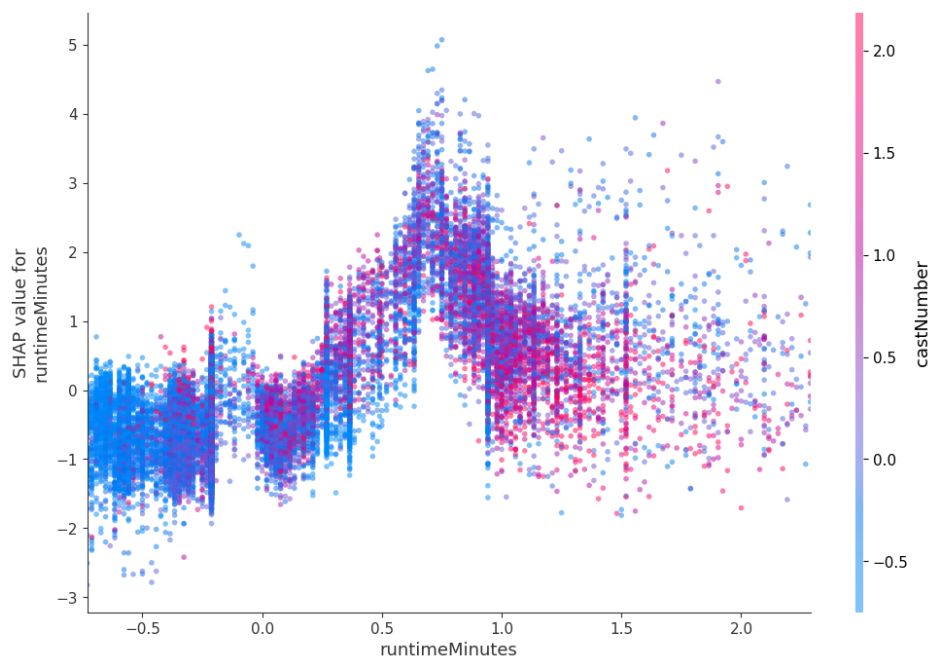


Figure 5.3: Dependence plot of runtimeMinutes with interaction on castNumber.

5.0.2 Local Explanations

Local explanations provide insights into why the model made a specific prediction for a single instance. These explanations are particularly useful when understanding misclassifications or borderline cases. For example,

Figure 5.4 shows the local feature attribution for one test sample. Features such as `writerCredits` and `averageRating` had a strong positive contribution towards the adult class, while `castNumber` and `word_count` strongly pushed the prediction towards the non-adult class. The final classification results from the balance of these opposing contributions.

In this case, the model correctly predicted the instance as non-adult because the strong negative contributions from `castNumber` and `word_count` outweighed the positive signals from `writerCredits` and `averageRating`. This demonstrates how individual-level explanations can highlight the reasons for a single prediction, which may differ significantly from global trends.

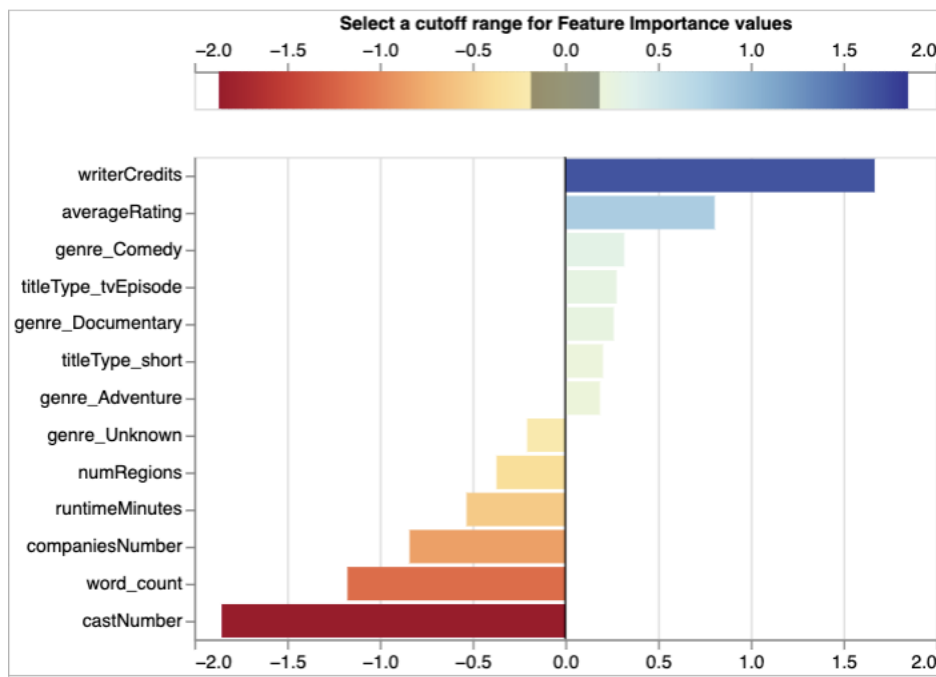


Figure 5.4: Local feature importance explanation for one instance.

5.0.3 Conclusion

The combination of global and local SHAP explanations provides a comprehensive understanding of the CatBoost model. Global analyses show that structural and temporal variables such as `total_multimedia`, `release_decade`, and `runtimeMinutes` play a central role in shaping predictions. At the same time, local explanations illustrate how specific feature values interact in individual cases, sometimes in ways that differ from the average behavior.

This dual perspective ensures that the model is both interpretable at a system-wide level and accountable at the level of single decisions. Such an approach increases trust in the model's predictions and enables practitioners to better assess its alignment with domain knowledge and ethical requirements.